

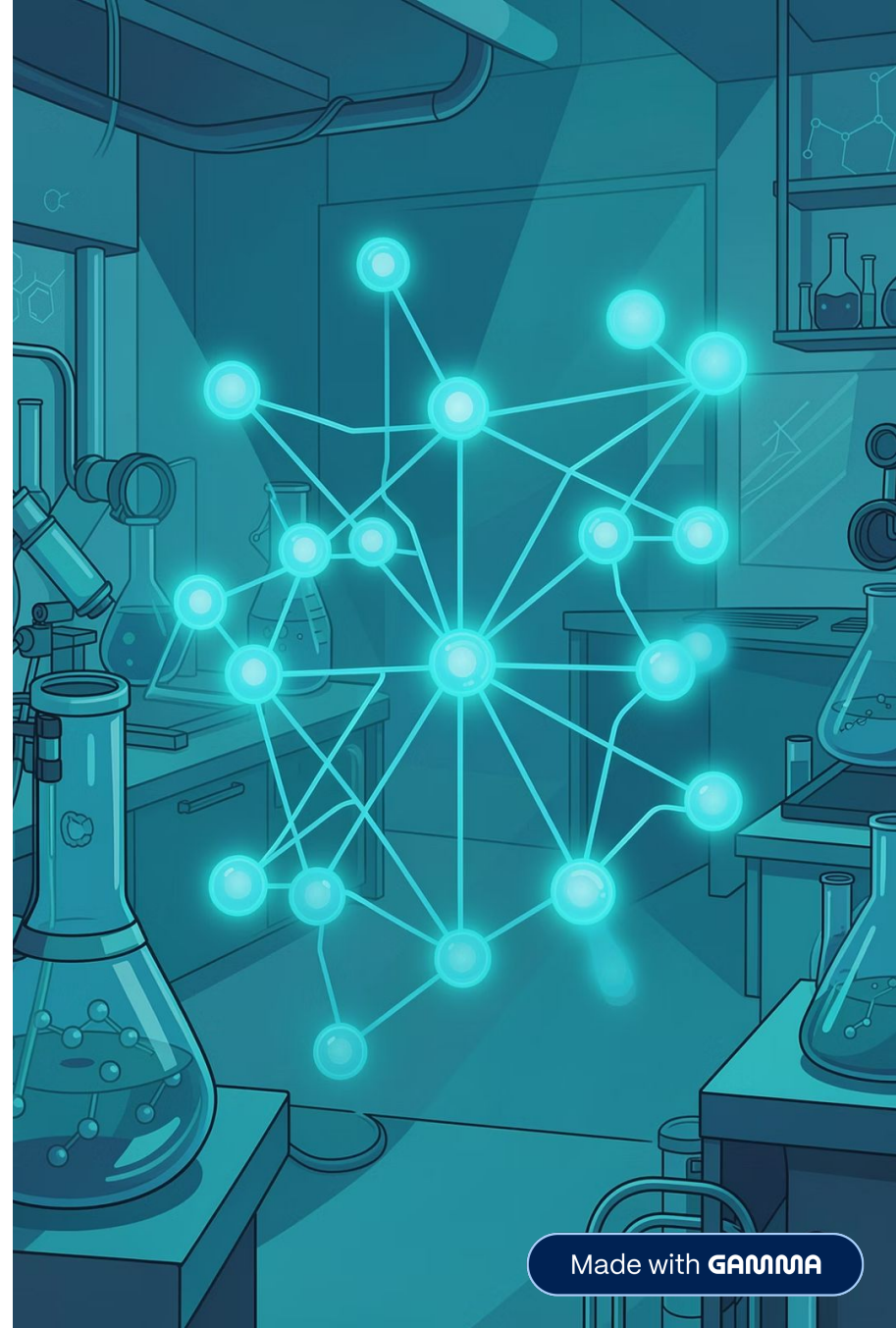
GraphDTA Benchmark Journey

A hands-on research walkthrough — from raw CSV data to a fully reproducible, multi-seed drug-target affinity benchmark built from scratch in Google Colab.

DEEP LEARNING

DRUG DISCOVERY

PYTORCH GEOMETRIC



Made with GAMMA

The Problem: Predicting Drug-Target Binding Affinity

Drug discovery is one of the most expensive and time-consuming endeavors in modern medicine. A critical bottleneck is determining how strongly a drug candidate binds to its protein target — a property known as **binding affinity**. Traditional wet-lab assays are slow and costly, making computational prediction methods essential for early-stage screening.

Why It Matters

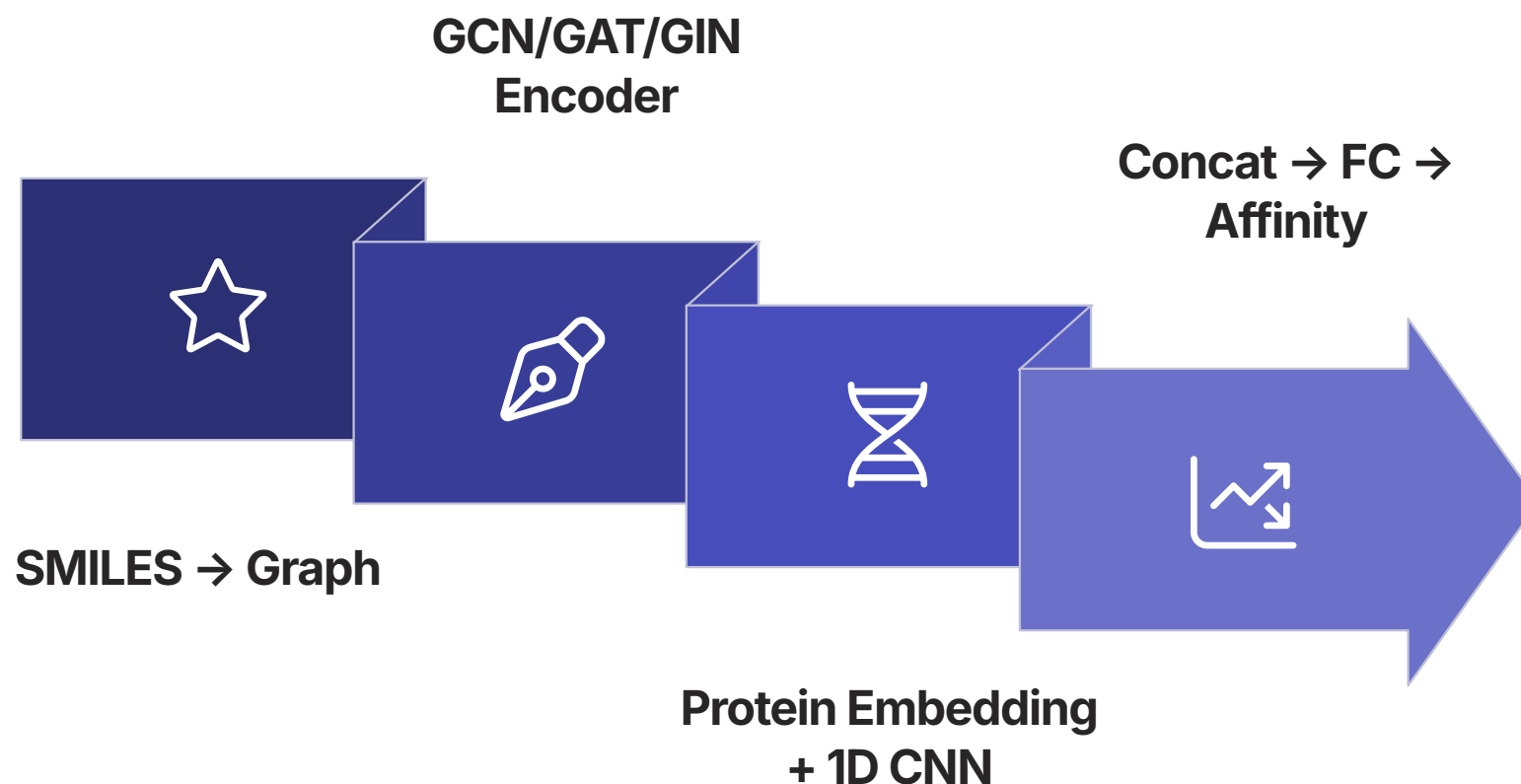
- Billions of drug-target pairs exist — impossible to test experimentally at scale
- High-affinity binding is a prerequisite for drug efficacy
- Faster predictions accelerate the drug development pipeline by years

The Gap GraphDTA Fills

- Earlier models (DeepDTA) treated drug molecules as raw SMILES strings — ignoring molecular topology
- GraphDTA encodes drugs as **molecular graphs**, capturing atom-level structure and bond relationships
- Result: richer drug representations leading to stronger predictive power

How GraphDTA Works

GraphDTA is an elegant two-branch neural architecture that jointly learns from molecular graph structure and protein sequence context, then fuses both representations to regress binding affinity.



Drug Branch

SMILES strings are parsed into atom-and-bond graphs. Each atom is a node with features (atomic number, degree, aromaticity, etc.). Graph Convolutional Networks — or GAT/GIN variants — aggregate neighborhood information to produce a fixed-size drug embedding.

Protein Branch

Protein sequences are integer-encoded, passed through a learnable embedding layer, then processed by a 1D CNN with multiple filter sizes. Global max-pooling extracts the most salient sequence features into a compact protein vector.



Our Project Goal: Fair, Reproducible Benchmarking

We set out to implement GraphDTA from the ground up in Google Colab — not just run a pre-trained checkpoint, but build the full pipeline ourselves so every design decision was transparent and auditable.

Goal 1: Data Fidelity

Convert our own CSV data into proper PyTorch Geometric graph datasets with atom-level features, protein encodings, and clean train/val/test splits.

Goal 2: Reproducibility

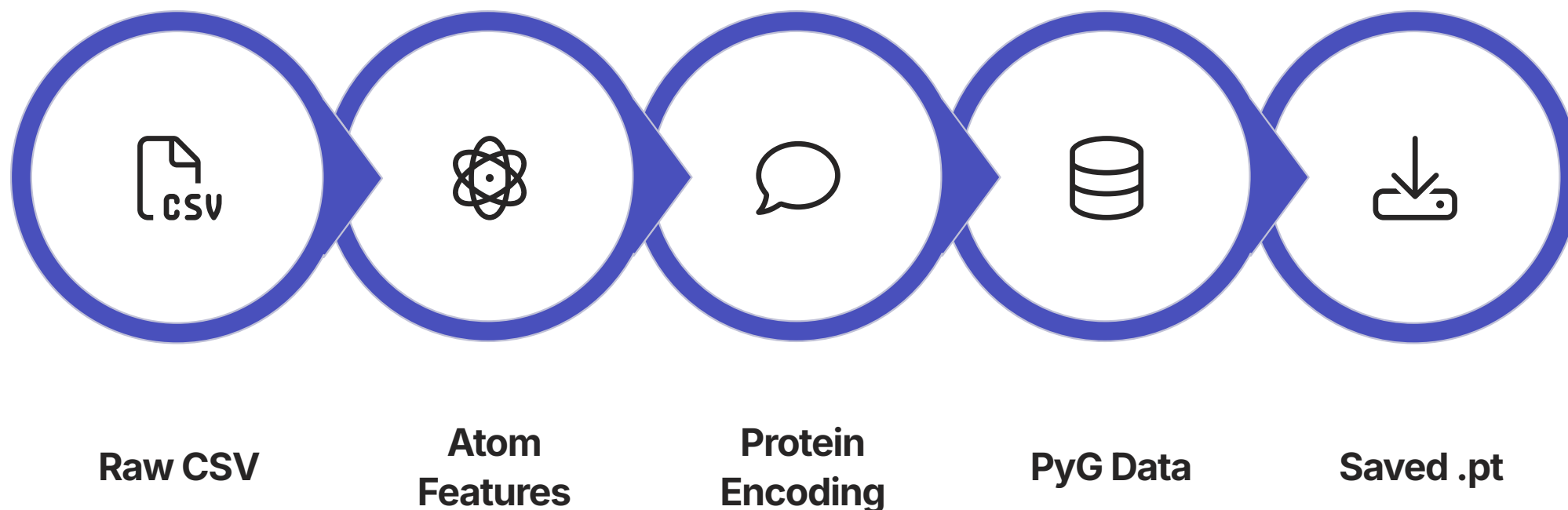
Fix a global random seed and evaluate across multiple seeds (32, 42, 52, 62, 72) to measure variance and confirm result stability.

Goal 3: Rigorous Evaluation

Apply bootstrap resampling (1,000 iterations per seed) to compute confidence-aware estimates of RMSE, R^2 , Pearson, Spearman, and CI.

Data Preprocessing Pipeline

Raw data arrived as CSV files with SMILES strings, protein sequences, and affinity labels. Every record needed to be transformed into a structured graph object before any model could touch it.



Drug Graph Construction

- RDKit parses each SMILES into an atom-bond graph
- Node features: atomic number, degree, formal charge, chirality, aromaticity, H-count
- Edge indices encode bond connectivity with bi-directional adjacency

Protein Encoding

- Amino acid sequence mapped to integer vocabulary
- Fixed-length padding/truncation applied (max 1000 residues)
- Stored alongside the drug graph as a single `torch_geometric.data.Data` object with label `y`

Training Pipeline Architecture

We implemented the full training loop using PyTorch Geometric, with careful attention to reproducibility controls, monitoring, and checkpoint management.



Dataset & DataLoader

Custom `GNNDataset` classes load processed `.pt` files. PyG DataLoaders batch heterogeneous graph objects efficiently with configurable batch size and shuffling.



GCN + CNN Model

Drug graphs pass through three GCN layers with ReLU and global max-pool. Protein sequences pass through an embedding layer and three 1D conv layers. Concatenated vectors feed into fully connected regression layers.



Early Stopping

Validation loss is monitored each epoch. Training halts automatically if no improvement is observed for a set patience window — preventing overfitting and saving compute.



Checkpointing & Logging

Best model weights are saved per seed. Training logs capture epoch-level train/val loss. Test predictions (`y_true`, `y_pred`) are exported as CSV for downstream bootstrap evaluation.

The Debugging Journey

No real project runs cleanly from line one. We encountered and methodically resolved three major categories of issues before training converged reliably.



Issue 1: Namespace Conflict with Local datasets Package

A locally named `datasets/` folder shadowed the PyTorch Geometric dataset imports. Python's module resolution picked our folder over the installed library. Fix: renamed the local directory and cleared stale `.pyc` caches, restoring clean imports.



Issue 2: NaN Affinity Labels in CSV

Several rows in the raw CSV contained missing or malformed affinity values that silently produced `NaN` tensors during batching, causing loss to explode. Fix: added an explicit NaN-drop step during preprocessing and re-generated all `.pt` files.



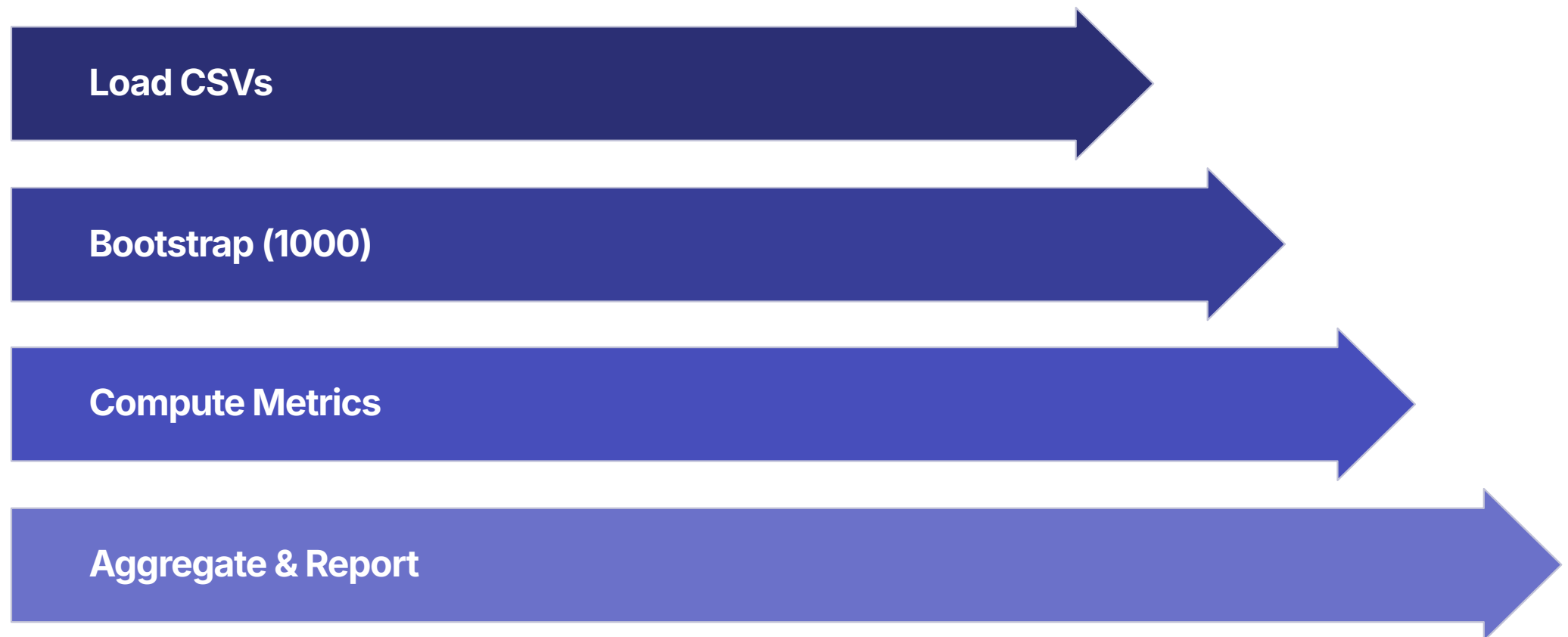
Issue 3: Wrong Root Paths for Processed Files

After reorganizing the repository structure in Colab, the dataset root paths pointed to stale locations, causing the loader to silently skip preprocessing. Fix: aligned all root paths with the actual Colab directory tree and force-regenerated processed splits.

- ✓ After resolving all three issues, training ran cleanly end-to-end — predictions were saved for every seed without interruption.

Evaluation Pipeline

Evaluation was designed to go beyond single-point metrics. We implemented a rigorous bootstrap resampling approach to quantify uncertainty across all reported statistics.



Metrics Explained

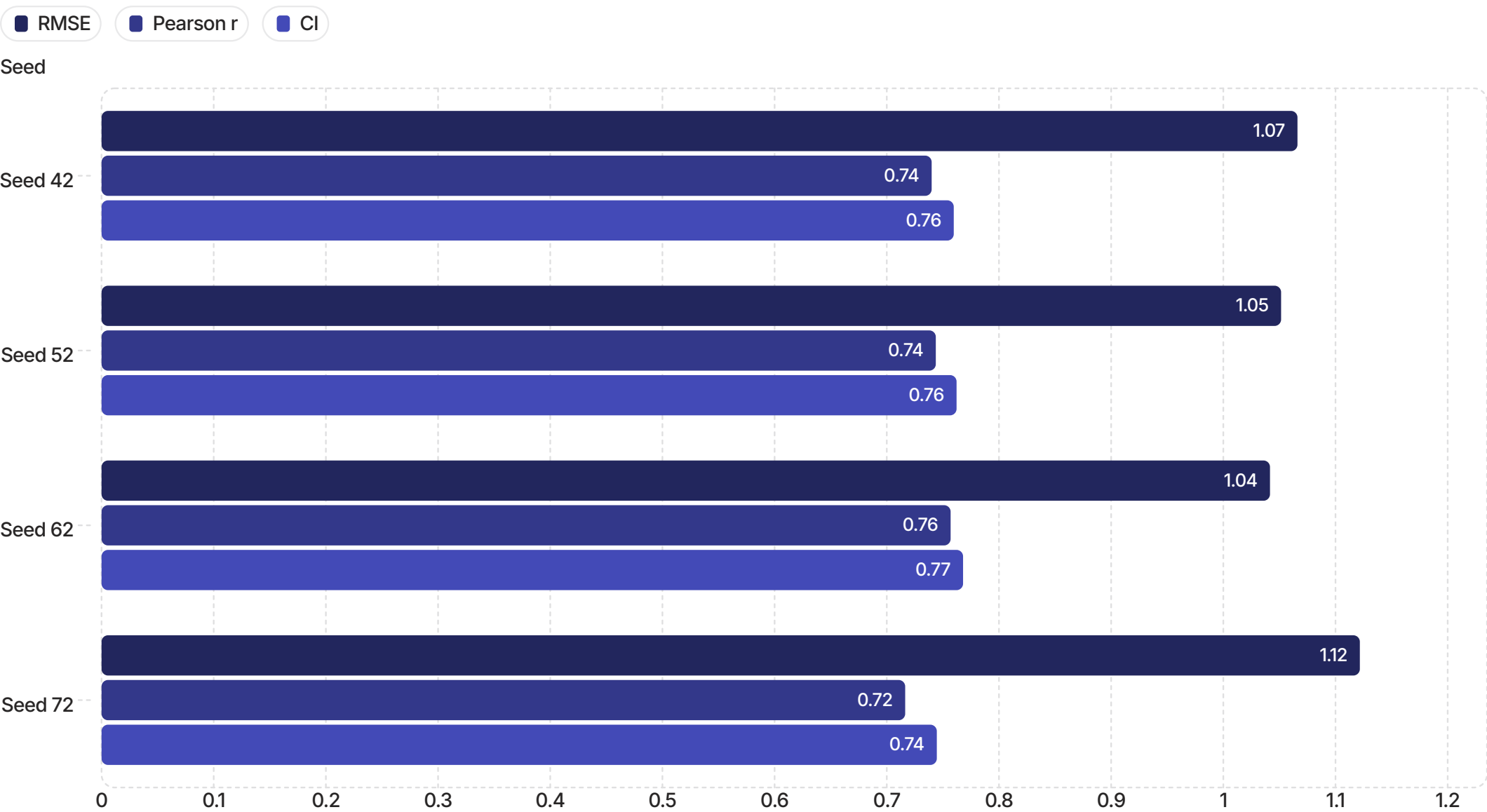
- **RMSE / MSE**: Error magnitude in affinity units
- **R²**: Variance explained by the model
- **Pearson r**: Linear correlation between predicted and true values
- **Spearman ρ**: Rank-order correlation, robust to outliers
- **CI**: Concordance Index — how well the model ranks pairs

Bootstrap Design

- 1,000 resamples per seed — each resample draws test-set size samples with replacement
- All six metrics computed on every resample
- Mean ± Std reported as the final estimate — more honest than a single test split
- Results saved per seed for cross-seed aggregation

Multi-Seed Results

We trained across five seeds — 32, 42, 52, 62, and 72 — to measure result stability. **Seed 62 delivered the strongest test performance** among the reported runs, while seed 72 was the weakest, revealing natural variance in stochastic training.



1.109

Mean RMSE

± 0.092 across all five seeds

0.715

Mean Pearson r

± 0.056 — strong linear correlation

0.749

Mean CI

± 0.024 — consistent ranking quality

0.510

Mean R²

± 0.079 — solid explained variance

Outcomes & Takeaways

From a blank Colab notebook to a clean, multi-seed, bootstrap-evaluated GraphDTA benchmark — here's what we learned and where we go next.

✓ Reproducibility Achieved

Seed control, checkpoint saving, and deterministic data splits made every result traceable. Any seed can be re-run and should match within bootstrap variance.

📊 Benchmark Quality

Bootstrap resampling with 1,000 iterations per seed gives honest confidence estimates rather than optimistic single-split numbers. Our reported metrics are audit-ready.

🧠 What We Learned

Namespace conflicts and NaN labels are silent killers in ML pipelines. Rigorous preprocessing and explicit path management are non-negotiable foundations of a clean experiment.

🚀 Next Steps

Extend to GAT and GIN drug encoder variants. Explore larger datasets (BindingDB, KIBA). Implement attention visualization to interpret which molecular substructures drive affinity predictions.

- 📄 Seed 62 achieved the best individual test performance: RMSE $\approx$ 1.042, Pearson $\approx$ 0.757, Spearman $\approx$ 0.715, CI $\approx$ 0.768 — setting a strong baseline for future comparisons.